Summarize

Chapter 15. AI Inference and Serving

In the last few years, AI has become a key part of many different types of applications. Though the basics of neural networks and machine learning have been around for decades, in the last decade advances in deep learning and large language models (LLMs) have created a phase shift in the quality of models and the applications that are possible for AI. More crucially, these systems have captured the imagination of application developers all over the world who see limitless ways to apply LLMs to their particular business domains.

AI and machine learning is a complex topic that can take years to master, but fortunately, with the assistance of libraries and pre-built models, it takes significantly less time to begin to incorporate intelligence into your application. This chapter does not attempt to make you an AI expert, but it can serve as an introduction to the concepts and approaches for using AI in your system.

The Basics of AI Systems

Before we get started on the details of using AI in your system, it is useful to get a grounding in the core concepts that make up AI application. The place that most people start is with a model. A model is a collection of numeric weights that encode the knowledge in a neural network. In modern LLMs, there are trillions of these weights. As a rough definition, you can think of the model as a function that takes a collection of inputs and transforms them into some output.

Unlike a traditional function in a programming language, however, a neural network model is learned via training. Training is the act of taking large amounts of training data and training the model from this training data. The training data contains pairs of both inputs and outputs. When training happens, the input is fed into the existing model, and its current output is observed. The current output is compared to the expected output in the training data. The error between the expected output and the observed output is fed back through the model in a single training iteration. This process is repeated over and

over again until the model correctly outputs the expected training outputs. This training process is computationally expensive and uses dedicated hardware.

Because of the cost and complexity of training these LLMs, more recently people have used fine-tuning to refine foundational models like ChatGPT. Fine-tuning is the process of taking a general-purpose model and refining it to better match your application. Fine-tuning generally uses smaller data sets and less computation so it is more efficient from a cost perspective.

Once you have a model trained, it can be put to use within your application. While training evaluated the model with known inputs and expected outputs, inference is the process of evaluating the model on user inputs where the output is unknown. Inference is typically implemented as a RESTful API, which you can call with the user's input.

Many cloud providers provide inference as a service. Such solutions can make it significantly easier to get started incorporating AI into your application and often allow access to models that may not be available for your own usage. However, some applications have data privacy or security requirements that mean they need to host inference on their own servers. We'll discuss how this is done in later sections.

For LLMs, the final part of inference is the prompt itself. When you chat with an LLM, it may seem as if your query text is sent directly to the model, but in most cases it is wrapped as part of a larger text that is called the prompt. The prompt provides context and instructions to the model that are not included in your specific query. For example, a prompt might add your location or other personalization information that is known about you to your query. The structure of the prompt is critical in getting good responses from an LLM. The act of creating a good prompt is known as prompt engineering. In addition to the examples above, prompt engineering can be used to add additional data from traditional data sources and to ensure that the responses from the model adhere to your application's requirements for accuracy and stability.

"The Basics of AI Systems" described the components of an AI application. The following sections will get into more details about how the application can be built.

Hosting a Model

Building a model is a large, complex task that could fill an entire book (or more) and is thus outside of the scope for this chapter. Even if you are interested in doing your own training or fine-tuning, you are encouraged to begin by using pretrained models so that

you can get a sense for how well AI works with your application. Not every application is a great fit for the current state of AI models, and thus, before investing the time, energy, and money in training, it makes sense to evaluate your overall approach.

When you are looking to host a model, the first thing to consider is the computational requirements of the model and your application. If your application is going to be used interactively by a user, the latency of the responses is critical to the quality of the application. Generally, users begin to notice latency at 250 milliseconds and start turning away and doing other tasks when latency reaches more than a few seconds. Thus, in interactive inference, performance is critical.

Typically, LLMs require high-performance dedicated GPUs to achieve the required latency. Such GPUs are expensive to purchase for your own hardware or rent from the cloud; further, given the intense interest in AI, the chips themselves have been hard to obtain from time to time. LLMs typically cannot run on the hardware profile of a mobile device such as a phone or tablet. If your application is intended for a mobile use, you will need to run the inference in the cloud and access it from the mobile device over the network.

The challenge of the cost of inference has led to the development so-called "small language models." These models, like Microsoft's Phi family of models, use much more focused training data to achieve reasonable performance with many fewer weights (billions versus trillions). Though their scores on AI benchmarks cannot compete with larger models like ChatGPT, the reduced size means that both the cost and latency of inference can be dramatically reduced. For many applications, the trade-off is worth it, and small language models (SLMs) can deliver acceptable quality at a much lower cost. Furthermore, since SLMs can run on a mobile device, they also can help address concerns from users who do not want to share their data with a cloud-based model.

Distributing a Model

So far we've talked a lot about models for both training and inference, but we've had little discussion of how the models themselves are actually stored to file and loaded for processing. There are several different frameworks for performing training and inferences (we'll discuss them in a coming section), and each framework has its own file format; for example, TensorFlow uses the TF2 file format. It's fine to use a framework-specific format if you don't plan to distribute your model and you don't expect to ever change formats. However, for most users a framework-independent open format for models is the best choice.

The most popular open model format is ONNX (Open Neural Network eXchange), sponsored by Microsoft, Facebook, and Amazon and used by thousands of open source developers around the world. Models that are stored in the ONNX format can be used in all of the popular frameworks for inference and learning and are supported by various model "hubs," or galleries of popular models.

For many users, especially when you are getting started, using someone else's open source model is the best way to start performing inference, much like cloud native container users start downloading container images from the Docker hub. These model hubs provide easy access to the most popular open source models. One of the most popular model hubs is called Hugging Face. Hugging Face includes not just the model distribution, but also code to make it very easy to download and start using the models themselves. Nearly the entire AI ecosystem uses Python as the language of choice. Getting started with a model from Hugging Face is as easy as a one-liner of Python code.

The models stored in these hubs can be quite large; and thus, while you can download them every time your application starts up, you probably don't want to, as it can add minutes of time to your application startup. Instead, the best pattern is to cache the model locally, either within a shared filesystem or an in-memory key-value store. Once you cache your model, all of your instances for inference can load from the same cached model. When you restart your application because of failures or new software releases, you don't have to delay to redownload the same model over and over again.

The final consideration with model distribution is safe deployment of new models. Though changes to models typically happen less frequently than changes to your code, you will make changes to the model over time. Just like safe deployment of your code, it is critical that you perform safe deployment of your model. Changes in your model can have significant and unpredictable impact on the quality and reliability of your application. To limit these risks, you should follow the best practice of progressive exposure and start by using the new model on only a very small subset of your users, typically tied to a percentage of users or a specific geography. As you gain confidence that the model is working correctly, you can gradually expand it to more and more users until ultimately all of your application is using the new model.

Development with Models

The complexity of inference means that it requires significant compute capacity. Whether you are hosting your own models or using a cloud-based model as a service, this means that every inference is expensive. Unless you are specifically doing develop-

ment related to the quality of your inference (e.g., prompt engineering), it probably doesn't make sense to pay the costs of a full model when you are working on the user interface or other parts of the service which are not directly related to the AI.

You should already be using a RESTful web API to perform inference, whether with services like Azure machine learning or models you host yourself. You can use this interface to hide the details of the specific model being used for inference and thus save costs when you don't need the full model.

In this approach, you develop a parameter to the API, which indicates which model to use. If you are using a cloud-based API, you may need to develop a wrapper RESTful service that provides this switch. When you deploy this compatibility interface, you choose which model to target. In development and testing environments, you can create a Kubernetes Service resource which points to a cheap SLM; whereas in production environments, you can use the full model. There are already several open source projects out there which provide generic inference APIs that can span a wide variety of different models.

Similarly, when performing continuous evaluation and automated unit testing of your application, it makes sense to deploy smaller, cheaper models, unless you are specifically evaluating quality.

Retrieval-Augmented Generation

Users interacting with AI systems expect that they are capable of understanding the context of the specific user, as well as the latest information available in the world. Unfortunately, because of the nature of AI models, the models themselves are fixed at a specific point in time when they were trained. Because these models are static and based on generic information, providing responses that are specific to the user or reflect the current state of the world are challenging. Retrieval-Augmented Generation (RAG) is a technique developed to handle these challenges.

With RAG, the user's prompt is augmented with traditional queries to provide the necessary context to correctly deliver an answer to the user's input. Consider, for example, the query:

> *Prompt: Who is the closest friend to my current location?*

It is clear that a generic LLM has no knowledge of either who the person's friends are, nor their current location. We need RAG to solve this problem.

When implementing a system with RAG, in addition to the user's prompt, the system performs two independent queries. The first might be a traditional database query across the user's contact list to determine the complete set of friends. The second query might be to their phone's location services to determine the user's current location.

The results of these two traditional queries are combined with the user's prompt and then sent to the model. An example RAG prompt might look something like:

> *Prompt: Given a set of friends with `${address}` and a current `${location}`, who is the closest friend to the current location?*

We can see how the personalized and up-to-date information provided by the RAG retrievals are combined with the user prompt to provide the LLM with sufficient information to correctly respond to the user.

RAG is a great technique for experienced developers who may not have lots of AI experience to develop sophisticated AI applications. By combining traditional database queries with natural language prompts from users, very successful natural language interfaces can be built that feel very personal. A further benefit of the RAG approach is that it respects user privacy. None of the user's personal information is encoded into the LLM, where it might accidentally be revealed to other users. Instead, the information is only accessible to the user that has access to the information, and provides it to the model in the form of an augmented prompt. Because of its explainability and privacy-protecting features, RAG is one of the most popular techniques for chat-based AI systems today.

Testing and Deployment

At this point, the idea of testing our services is well understood and accepted as a required practice if you want to maintain a reliable and well running service. But what does it mean to perform a unit test in the context of an AI application when making changes to the prompt can have significant impact on the different responses that you get from the model?

The first important mind-shift that must be made when testing AI applications is the switch from correctness testing to statistical at-scale quality testing. When we were taught to write unit tests, we were taught to evaluate whether the output of the function that we were testing was correct. Indeed, there are whole frameworks dedicated to expressing the expectation that `Value(A).equals(B)`. Such tests do not work for AI applications.

Instead of looking for exact matches, we are looking for our tests to evaluate if a response is "good enough." Once we shift our minds from "correct" to "good enough," we realize that evaluating a single input is no longer acceptable. As the model changes, the response to any one input may change dramatically. Indeed, while a change may make the response to one type of input dramatically better, it may make the response to many other inputs dramatically worse.

Thus, to correctly test our AI applications, we have to consider their performance across a large selection of prompts and evaluate if, on average, the change improves or harms the quality of the responses. If a change has zero or positive impact on the overall quality of the responses, the change passes the test. If the quality of the responses is decreased, the test fails.

But then to actually implement such a test, we need a way to assess the quality of a response. This task itself can be a challenge. Obviously, it is far too expensive to have humans evaluate the quality of each response—some automated solution is required. Fortunately, LLMs themselves can be used for some of this evaluation. Just like you used the model to answer the user's input, you can likewise ask the model if the answer it gave is a high-quality answer. Your prompt looks something like:

> *Prompt: Given the following input:* `${input}` *is the response:* `${response}` *a high-quality response?*

This approach may seem like giving the inmates control of the asylum, but it actually works well in many cases.

In addition to such AI-driven quality testing, before you release a new code or prompt change, it is critical that you incorporate user feedback signals into your safe deployment. While, in other systems, you may look at errors and latency to determine if a release is safe to continue, when rolling out an AI application, you must also measure and evaluate if your users continue to be happy with the responses they are receiving.

AI changes the way we build applications, and thus, also changes the way those applications are tested. However, it does not change the criticality of testing and evaluating every change you make to ensure it does not impact the overall quality of your application.

Summary

In just a few years, AI has transformed both our imagination of what our applications could be, as well as our users' expectations for how they can interact with those applica-

tions. However, at the end of the day, AI inference is just another function that is provided by a distributed system and delivered to our users. This chapter provides an introduction to the core concepts and techniques for building AI applications that will enable anyone familiar with distributed systems to augment their distributed applications with AI.

Previous chapter
< 14. Monitoring and Observability Patterns

Next chapter
16. Common Failure Patterns >